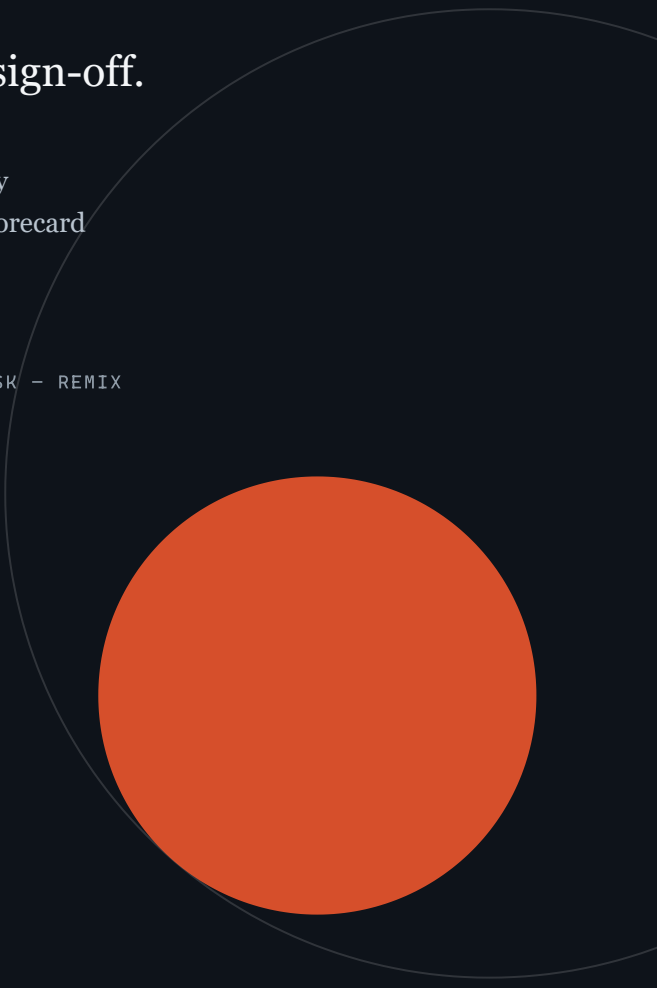
FLIP THE SWITCH HACKATHON · BUILD PLAN

Signoff

One engine. Three verticals. One human sign-off.

A Switch-native compliance room that turns a requirement-heavy document into a compliance matrix, a cited draft and a review scorecard — with a person holding the final decision.

01 GRANT ROOM — HERO 02 BID ROOM — REMIX 03 DEAL DESK — REMIX



TRACK

Track 01 · Best Business Workflow
$2,000

DEADLINE

Nov 7, 2026 · 11:59 PM IST
Internal target: 6 PM IST

TEAM

2–4 people
Plus three Switch agents

PLATFORMS

Discord primary
Slack second connection

STACK

Switch · OpenCode / Claude Code
Python 3.13 toolbelt

REPO

github.com/varun2507027108-oss/signoff-switch

00 At a glance

We are building **Signoff**: a reusable compliance workflow that runs inside a Switch room. The hero demo is a nonprofit turning a funder's guidelines into a submission-ready grant application. The same engine also runs an RFP response and an agency quote-to-kickoff flow.

THE PROBLEM

Grants, RFPs and client briefs are slow, repetitive and easy to lose on a missed attachment or word limit. Small teams have no compliance person.

THE PRODUCT

Three agents turn a source document into a compliance matrix, a cited draft and a review scorecard. A human makes the call.

WHY SWITCH IS CENTRAL

Two linked rooms carry intake and review. Per-agent addressing decides who can wake each agent. The room history is the audit trail.

THE DIFFERENTIATOR

A deterministic checker validates citations, word limits, attachments and rubric coverage. Agents cannot invent evidence unnoticed.

THE DEMO

A grant deadline, a missing audit, a 78/100 review, a human decision and a final DOCX — then a 20-second Bid/Deal montage.

THE SUBMISSION

Public repo, 3-minute video, README, template YAML, sample packs and tests — plus a blog post and a #HowISwitch clip.

What we are entering

ENTRY	PRIZE	OUR PLAN	PRIORITY
Track 01 — Best Business Workflow	$2,000 cash	Signoff, with Grant as the hero demo	PRIMARY
Best Blog	AirPods 5	"Building Signoff with Switch" on Dev.to, 1,200–1,800 words	HIGH
#HowISwitch	Logitech keyboard (top 5)	Short post with a 20-second clip and the repo	EASY
Track 03 — Best Core Contribution	$100 × 20 PRs	Skipped as primary; issues are connector-sized and heavily commented	SKIP
Code with Wispr	$500	Only if you already use Wispr Flow and can submit the Insights scorecard	CONDITIONAL

Definition of done

- **It runs:** a judge can clone the repo, install the toolbelt, run the sample, and see the same artifacts without an LLM.
- **It is Switch-native:** the live workflow uses a group template, three agents, two rooms, a link, room instructions, a shared workspace, and per-agent addressing.
- **It is presentable:** a 3-minute video with captions, a README with screenshots and a "Judge's 5-minute path," and a published blog post.
- **It is submitted:** repo, video and README submitted before **6 PM IST on Nov 7**, with buffer before the 11:59 PM deadline.

01 The event constraints

WHAT THE JUDGES AND RULES ACTUALLY REQUIRE

3.5	$10k	4	1
DAYS LEFT AT PLAN TIME	TOTAL PRIZE POOL	JUDGING CRITERIA	SUBMISSION PER PROJECT

Dates and format

ITEM	DETAIL	IMPLICATION FOR US
Hacking window	Nov 4–7, 2026	Today is setup + schema day, not content day.
Submission deadline	Nov 7, 11:59 PM IST	Internal deadline: **6 PM IST**, six hours of buffer.
Team size	Solo or up to 4	We have 2+ humans for the room and recording.
Required artifacts	Repo, demo video, README	All three are first-class build outputs, not afterthoughts.
Switch requirement	Switch must be central	The workflow must stop without Switch, not just "use it once."
AI tools	Allowed	We can use OpenCode/Claude Code to build the project.

Tracks and prizes

TRACK	PRIZE	WHAT WINS	OUR READ
01 · Best Business Workflow	$2,000	Most practical workflow where people and agents get work done	Our primary. Grant/RFP compliance is a real, expensive team problem.
02 · Personal or Creative	$2,000	Everyday tasks, personal projects, creative ideas	Not our primary; the business artifacts are stronger.
03 · Core Contribution	$100 × 20	Top 20 useful, well-built PRs to Switch core	Skipped. Open issues are mostly connectors with 10–25 comments; assignment is competitive.
Blog	AirPods 5	Clarity, accuracy, usefulness to a first-time Switch user	High value for ~2 hours of work. We will enter.
#HowISwitch	Keyboard (top 5)	Social post sharing the build	10 minutes. We will enter.
Code with Wispr	$500	Best project built by voice, with Insights scorecard	Only if you already use Wispr Flow. Do not fake usage.
Swag draw	20 projects	Random draw among submissions	Submitting enters us automatically.
Certificate	Everyone	Every submission	Automatic.

How the judges score

USE OF SWITCH

Is Switch central, and would the project stop working without it?

TECHNICAL EXECUTION

Does it run, and could someone clone the repo and reproduce it?

CREATIVITY AND IMPACT

Is it a fresh idea that solves a problem a real team has?

PRESENTATION

Do the demo video and README make clear how Switch is used?

STRATEGIC READ

With roughly 7,000 registrants, the winning entry cannot be "an AI bot in Slack." It has to show a workflow that is visibly better because the agents share a room, hand off work, respect permissions, and leave an audit trail. Our edge is the **deterministic checker** plus **real, inspectable artifacts**.

Rules that affect the build

- **Track 03 only:** issues labelled `hackathon` count, one person per issue, assignment required before code. We are not relying on it.
- **No bots or automated entry:** the submission and registration must be the human team's. AI tools are allowed in the project itself.
- **Original work:** no copied proprietary code, no secrets in the repo, no third-party confidential data. The sample funder and nonprofit are fictional.
- **Open-source licenses:** our repo can be MIT; we use Switch as a dependency, not a fork.
- **Demo clarity:** the video must show Switch being used, not just the final output.

What we optimize for

CRITERION	WHAT THE JUDGE IS ASKING	EVIDENCE WE PUT IN FRONT OF THEM
Use of Switch	Would this stop working without Switch?	A group template creates three agents, two rooms and a link; per-agent addressing; the room history is the audit trail.
Technical execution	Can I clone it and reproduce it?	Fresh-clone quickstart, deterministic `signoff demo`, pytest + CI, three sample packs.
Creativity and impact	Is it fresh, and does a real team need it?	Grant compliance is a real cost; the same engine proves Bid and Deal reuse in the same video.
Presentation	Do I understand it in three minutes?	A timed demo with the human decision, the checker catch and the final DOCX; a README "Judge's 5-minute path."

WHAT WOULD MAKE US LOSE

A generic "AI bot in Slack" demo; a workflow that only runs on the presenter's machine; a polished UI with no real artifacts; or a video that never shows the human decision. Every one of those is a design constraint, not a presentation problem.

The product: Signoff

Signoff is a Switch-native compliance room. Drop in a requirement-heavy document, and three agents turn it into a compliance matrix, a cited draft and a review scorecard. You make the judgment calls and sign off. Every decision stays in the room.

The problem in one paragraph

A small nonprofit applying for a grant, a B2B team answering a 40-page RFP, and a design studio scoping a client project all do the same kind of work: read a long document, extract every requirement, find the evidence, write a response, check it against a rubric, and get a human to approve it. The work is slow, repetitive, and easy to lose to a missed attachment or an uncited claim. The people who do it are smart; the process is the bottleneck.

The workflow

CLIENT · INTAKE ROOM
Drops the guidelines and evidence pack. Answers two or three clarifying questions.

→

ANALYST · INTAKE + DRAFT
Builds the compliance matrix. Flags disqualifiers, missing attachments and word limits. Posts the handoff.

→

DRAFTER · DRAFT ROOM
Writes section by section. Cites evidence `[E#]`. Marks `[GAP]` instead of inventing.

→

REVIEWER · DRAFT ROOM
Scores against the rubric. Runs the checker. Red-teams the draft. Reports to the approver only.

→

APPROVER · BOTH ROOMS
Makes the judgment call. Approves the fixes. Triggers the export. Only the human can sign off.

One engine, three verticals

VERTICAL	INPUT	OUTPUT	EXTRA TOOL	DEMO ROLE
Grant Room	Funder guidelines + evidence pack	Compliance matrix, cited application, review scorecard, submission checklist	Eligibility flags	HERO · 3 MIN
Bid Room	RFP + approved answer library	Compliance matrix, drafted response, bid/no-bid memo, scorecard	Bid scoring	MONTAGE · 20 SEC
Deal Desk	Client brief + rate card + past projects	Scope, quote, invoice, kickoff pack, review	Pricing + margin	MONTAGE · 20 SEC

Why this is a fresh Track 01 entry

- **Not a docs example:** Switch's own use cases cover incident response, onboarding, vendor review and OSS review. Grant/RFP compliance is adjacent but distinct.
- **Real artifacts:** a judge can open the matrix, the DOCX, the scorecard and the decision log. The work is not trapped in a chat transcript.
- **Human judgment is the product:** the agents surface the disqualifier; the human decides to use the fiscal sponsor's audit. That is the moment the room earns its keep.
- **Generalizes visibly:** Bid and Deal reuse the same three agents and checker. The demo proves it is an engine, not a one-off prompt chain.

The 3-minute demo story

TIME	BEAT	WHAT THE VIEWER SEES
0:00–0:20	Hook	"A small nonprofit has three weeks to file a grant. Miss one eligibility rule and the application is dead."
0:20–0:40	Meet the room	Discord: `#grant-intake` and `#grant-draft`, three agents, two humans, one link.
0:40–1:20	Intake	Client drops the guidelines. Analyst posts the matrix and flags: audited financials required, only a fiscal sponsor letter in evidence.
1:20–1:50	Human call	Approver: "Use the fiscal sponsor's audit, E7. Proceed." Analyst posts the handoff into the draft room.
1:50–2:20	Draft + review	Drafter writes with citations and two `[GAP]` markers, posts "draft ready." The approver wakes the reviewer; the checker finds an uncited outcome and a missing attachment. Score: 78/100.
2:20–2:45	Sign-off	Approver approves fixes. Drafter exports the DOCX. The decision log records who approved what, when.
2:45–3:00	Proof of reuse	20-second montage: Bid matrix + bid/no-bid memo; Deal quote + invoice. Same agents, same checker.

Production plan

SEGMENT	HOW WE CAPTURE IT	IF IT FAILS ON THE DAY
0:00–0:40 Hook + room	1080p screen recording of the Discord window at 110% zoom; cursor highlight on; close personal DMs.	Use the static room screenshot and record the hook as voiceover.
0:40–1:50 Intake + human call	Full-window capture; teammate A posts from their own device so the client is visibly a second person.	Re-run with the seeded messages already in the channel; cut the waiting time.
1:50–2:45 Draft + review	Split screen: draft room on the left, terminal running `signoff check` on the right.	Cut to the recorded deterministic `signoff demo` output; narrate the same finding.
2:45–3:00 Proof of reuse	Four stills — Bid matrix, bid/no-bid memo, Deal quote, invoice — with a 2-second push on each.	Use the generated artifacts as stills; they are the proof, not the motion.

> RECORD THE CATCH, NOT JUST THE WIN
>
> The demo is stronger when the checker visibly fails something. Seed one uncited outcome and one missing attachment so the reviewer has a real finding. If the run is too clean, the audience cannot see why the guardrail matters.

03 Why Switch is central

THE WORKFLOW WOULD STOP WITHOUT IT

Signoff is not a Telegram bot with a PDF prompt behind it. The room is the workspace: it carries the client conversation, the internal review, the shared evidence, the permissions and the audit trail.

SWITCH PRIMITIVE	HOW SIGNOFF USES IT	WHAT BREAKS WITHOUT IT
Rooms	`#grant-intake` for the client-facing conversation; `#grant-draft` for the internal draft and review.	The client would see the messy internal draft, or the team would lose the separation between intake and review.
Agents	Analyst, Drafter and Reviewer are separate agents with distinct jobs and outputs.	One general-purpose bot would blur responsibilities and make the review meaningless.
Addressing	The client can only wake the Analyst. The Drafter takes handoffs from the Approver or the Approver's agents. The Reviewer only takes instructions from the Approver.	Anyone could wake the Reviewer or ask the Drafter to skip the checklist. The human gate would be advisory, not enforced.
Shared context	Room instructions carry the vertical brief; the shared workspace holds the evidence index and rubric. Every agent reads the same source of truth.	Each agent would need its own prompt with duplicated rules; updates would drift.
Links	The intake room points at the draft room with a "requirements handoff" label. The Analyst is the bridge and posts the structured handoff.	The draft room would have no signpost back to the source; context would be lost or manually pasted.
Room history	The decision log, the matrix, the review and the approval all sit in the room timeline.	No audit trail. A judge or a compliance officer could not reconstruct who decided what.
Templates	One group template creates the agents, rooms and link; the form's Existing agent choice lets Bid and Deal reuse the same three agents.	Every run would be manual setup, and the "clone and reproduce" claim would be false.

THE TEST WE WANT THE JUDGES TO APPLY

Delete Switch. Could you rebuild this with a chat app, three scripts and a shared folder? You would have to rebuild the room model, the per-agent permissions, the handoff, the shared context and the audit trail. That is the point: Signoff is not wrapping Switch; it is *made of* Switch.

The permission model, in one table

Each agent's addressing policy decides who can make it respond. Two guarantees are structural: the client is not in the draft room, and only the approver can wake the reviewer. The rest of the chain is a convention the agents are instructed to follow.

WHO	CAN WAKE	CANNOT WAKE	WHY
Client / program director	Analyst	Drafter, Reviewer	Member of intake only; the draft room is not visible.
Approver (you)	Analyst, Drafter, Reviewer	—	Owner of all three; holds the final sign-off.
Analyst	Drafter (handoff)	Reviewer	`owner-agents` admits the owner's agents; the reviewer is owner-only.
Drafter	Analyst	Reviewer	The analyst is `anyone`; the reviewer is owner-only.
Reviewer	Analyst, Drafter	—	Policy permits it; its instructions are to report, not direct.

Implementation: Analyst `anyone`, member of both rooms; Drafter `owner-agents`; Reviewer `owner`; client a room `user` in intake only. Switch v1 addressing is per-agent, not per-pair, so "only the approver wakes the reviewer" is policy; "the reviewer never directs the drafter" is instruction. The README states both.

Failure modes this prevents

SCENARIO	WITHOUT PER-AGENT ADDRESSING	WITH SIGNOFF'S SETTINGS
Client asks the Drafter to "just send the draft"	The Drafter is addressable by anyone in the room and leaks the internal draft.	The client is not a member of the draft room; the Drafter only takes handoffs from owner-agents.
Teammate asks the Reviewer to approve	The Reviewer rubber-stamps on request; the check becomes theater.	The Reviewer only accepts instructions from the approver, and it reports rather than approves.
An auditor asks who changed the eligibility answer	The history is scattered across DMs, docs and chat; no reliable record.	The room timeline and decision log show the analyst's flag, the approver's call and the reviewer's score.

THE GATE IS STRUCTURAL

The approver is a human room member, not an agent. Approval is a human message in the room, not an agent action: the reviewer scores, the approver decides. That is why the final click in the demo is a person's, not a model's.

Repository shape

```
signoff/
  signoff.template.yaml          # v1 group template: agents + rooms + link
  verticals/
    grant/                       # guidelines.pdf, evidence/, rubric.json, README
    bid/                         # rfp.pdf, answer-library/, rubric.json, README
    deal/                        # brief.md, rate-card.csv, rubric.json, README
  agents/
    analyst.md                   # source for the inline template text
    drafter.md
    reviewer.md
  src/signoff/
    cli.py                       # Typer entrypoint
    extract.py                   # PDF/HTML/MD -> text
    matrix.py                    # requirements schema + CSV/MD render
    checker.py                   # citations, limits, attachments, coverage
    rubric.py                    # weighted scoring
    export.py                    # DOCX / PDF
    pricing.py                   # Deal Desk quote + invoice
  tests/
    test_checker.py
    test_matrix.py
    test_rubric.py
    test_export.py
  demo/
    script.md                    # shot list + narration
    shots/                       # screenshots for README/blog
  README.md
  SUBMISSION.md
  LICENSE
```

Design rules

- **The checker never calls an LLM.** It is deterministic, offline and fast enough to run after every section.
- **Schemas are the contract.** Agents write JSON; the tools validate, render and score it. No agent writes the final DOCX directly.
- **Artifacts are files.** Every step leaves a CSV, Markdown, DOCX, PDF or JSON file in `out/`, so a judge can inspect the work without replaying a chat.
- **One vertical at a time.** Grant is complete before Bid or Deal gets a sample pack; the template's `vertical` param keeps the packs separate.

> WHY A CLI, NOT A WEB APP
>
> The visible product is the room. A CLI keeps the build small, makes the checker easy to test and gives the agents a stable interface they can call from any room. The web surface would add days of work without improving the demo.

The template

```
version: 1
params:
  vertical: { type: enum, enum: [grant, bid, deal], default: grant }
  bridge:   { type: bridge, default: [Discord, Slack, $first] }
  provider: { type: provider, default: [opencode, claude, codex] }
  workspace:{ type: directory, default: "{$agents_dir}/signoff" }
  client:   { type: user, label: Client username }

group:
  name: "Signoff {vertical} — {client}"

agents:
  - name: signoff-analyst
    display_name: Signoff Analyst
    description: Turns requirements into a compliance matrix and flags disqualifiers.
    provider: "{provider}"
    location: local
    directory: "{workspace}"
    addressing: anyone
    allow_existing: true
    instructions: |
      The room instructions name the vertical. Read
      verticals/<vertical>/README.md, the source document and the evidence index.
      Produce out/<vertical>/requirements.json, then run `signoff matrix`.
      Flag every disqualifier and missing attachment before drafting starts.
      Do not write prose. Hand off in the draft room, addressing
      @signoff-drafter with the matrix and evidence IDs repeated in full.

  - name: signoff-drafter
    display_name: Signoff Drafter
    description: Writes the response with evidence citations and explicit gaps.
    provider: "{provider}"
    location: local
    directory: "{workspace}"
    addressing: owner-agents
    allow_existing: true
    instructions: |
      You are the Signoff drafter. Use only evidence in the evidence index.
      Cite every factual claim as [E#]. If evidence is missing, write [GAP: ...].
      Respect every word limit. Run `signoff check` after each section.
      When the draft is ready, post one line addressed to the approver.
      Do not address the reviewer; only the approver can wake it.

  - name: signoff-reviewer
    display_name: Signoff Reviewer
    description: Scores the draft against the rubric and red-teams it.
    provider: "{provider}"
    location: local
    directory: "{workspace}"
    addressing: owner
    allow_existing: true
    instructions: |
      You are the Signoff reviewer, woken by the approver. Read the draft
      room history and the draft file. Run `signoff check` and `signoff score`.
      Score every rubric criterion. Find unsupported claims, missing requirements
      and weak answers. Do not edit the draft. Report to the approver only.

rooms:
  - name: "{vertical}-intake"
    bridge: "{bridge}"
    channel_type: channel_private
    instructions: |
      This room turns a source document into a compliance matrix.
      The client talks to the analyst. The analyst owns the matrix.
      Do not draft the response here. Hand off to the draft room.
    agents: [signoff-analyst]
    users: ["{$creator}", "{client}"]
    kickoff: |
      @signoff-analyst introduce yourself in two lines and tell the client
      what you need to start.
  - name: "{vertical}-draft"
```

```
    bridge: "{bridge}"
    channel_type: channel_private
    instructions: |
      This room drafts and reviews the response. The drafter owns the draft.
      The reviewer scores it. The approver makes the final call. Only the
      approver wakes the reviewer; no agent approves its own work.
    agents: [signoff-analyst, signoff-drafter, signoff-reviewer]
    users: ["{$creator}"]
    kickoff: |
      @signoff-drafter @signoff-reviewer this is the draft room. Drafter, say
      hello and describe what you will produce. Reviewer, say hello and describe
      what you will check. Wait for the analyst's handoff.
links:
  - from: "{vertical}-intake"
    to: "{vertical}-draft"
    label: "requirements handoff"
```

Point `workspace` at a clone of the repo, not an empty folder: the agents need `verticals/` and the CLI. `allow_existing` is a form choice, not automatic reuse; pick the same three names for Bid and Deal.

Agent roles

AGENT	OWNS	ADDRESSING	ROOMS	OUTPUT
`signoff-analyst`	Requirements matrix, evidence index, disqualifiers	`anyone`	Intake + draft	`requirements.json`, `compliance-matrix.csv`, handoff summary
`signoff-drafter`	Draft, citations, gaps	`owner-agents`	Draft	`draft.md`, `draft.docx`
`signoff-reviewer`	Rubric score, red-team findings	`owner`	Draft	`review.md`, `scorecard.md`
Approver (human)	Judgment calls, final sign-off	Owner	Intake + draft	Decision log, approval
Client (human)	Source document, clarifications	Room member	Intake	Guidelines, answers

Toolbelt

COMMAND	WHAT IT DOES	WHY IT MATTERS
`signoff extract`	PDF/HTML/MD → plain text + page map	Gives the analyst a stable source with page references.
`signoff matrix`	Validates `requirements.json`; renders CSV/Markdown	One schema for the analyst, drafter, reviewer and checker.
`signoff check`	Citations, word limits, attachments, coverage, uncited numbers	The deterministic guardrail against invented evidence.
`signoff score`	Weighted rubric from `review.json`	The reviewer cannot hand-wave a score; the tool computes it.
`signoff export`	Markdown → DOCX/PDF with styles	The approver gets a submission-ready file, not a chat reply.
`signoff price`	Rate card → quote + invoice + margin	Deal Desk's extra tool; reuses the same checker.
`signoff demo`	Runs the deterministic sample end-to-end	A judge can see the artifacts in seconds, without an LLM.

Checker rules

1. Every `[E#]` citation exists in `evidence/index.json`.
2. Every numeric or date claim has a citation or a `[GAP]` marker.
3. Every mandatory requirement in the matrix has a matching section/answer or a `[GAP]`.
4. Every section is within its word limit.
5. Every required attachment is present.
6. Every rubric criterion is addressed by at least one section.
7. Requirement IDs are unique and no mandatory row is missing.
8. The output is JSON + Markdown with file/line references, so the reviewer can quote the exact violation.

> THE GUARDRAIL THAT MATTERS
>
> If the drafter writes "we served 1,200 people last year" without a citation, the checker fails the draft. If it cites `[E7]` and E7 is a fiscal sponsor letter, the checker passes the citation but the reviewer flags the mismatch. The agents do the language; the tool enforces the evidence.

Data contracts

The agents exchange files, not vibes. Every handoff has a schema, so the next agent and the checker read the same fields.

FILE	KEY FIELDS	OWNER	CONSUMED BY
`requirements.json`	`id`, `section`, `requirement`, `mandatory`, `source_page`, `evidence_ids`, `status`	Analyst	`matrix`, `check`, Drafter
`evidence/index.json`	`id`, `type`, `title`, `path`, `date`, `issuer`	Analyst + client	Drafter, `check`, Reviewer
`review.json`	`criterion`, `weight`, `score`, `rationale`, `finding_ids`	Reviewer	`score`, Approver
`decision-log.md`	`actor`, `action`, `timestamp`, `artifact`	Approver	Export, audit trail

Build order

1. **Schemas first:** `requirements.json`, `evidence/index.json` and `review.json`. No agents yet.
2. **Checker and demo:** make `signoff demo` produce a matrix and a scorecard from seeded data.
3. **Template:** import the group template into a clean workspace; verify the agents, rooms and link.
4. **Grant live:** connect Discord and run the real conversation with the teammates.
5. **Bid and Deal:** add thin packs and the pricing tool; reuse the same agents and checker.

> WHY THIS ORDER
>
> Every step ends in something demoable. If the LLM provider or a bridge fails, the deterministic path still proves the product, and the demo has a fallback that does not depend on the network.

Grant Room — the hero demo

Persona: Maya, program director at a small workforce-development nonprofit. **Funder:** the fictional Regional Workforce Resilience Fund. **Deadline:** three weeks. **Stakes:** one missed eligibility rule kills the application.

ELEMENT	DETAIL
Input	12-page funder guidelines PDF + evidence pack (annual report, outcomes CSV, budget, staff bios, fiscal sponsor letter, audit, letters of support).
Requirements	6 narrative sections, 2 mandatory attachments, 4 eligibility criteria, rubric weights: need 30%, approach 30%, capacity 20%, budget 20%.
Agent outputs	Compliance matrix; cited draft; review scorecard; submission checklist; decision log.
Human moment	The analyst flags "audited financials required; only a fiscal sponsor letter in evidence." The director decides to attach the sponsor's audit (E7).
Demo result	Draft scores 78/100; reviewer finds one uncited outcome and one missing attachment; approver signs off after fixes; DOCX exported.
Before/after	Two days of grant admin → a 15-minute review. One disqualifier caught before submission.

Bid Room — the commercial remix

Persona: a B2B sales engineer answering a 40-page RFP. **Input:** RFP + approved answer library + case studies. **Output:** compliance matrix, drafted response, bid/no-bid memo, scorecard.

- **What changes:** the evidence pack becomes an answer library; the rubric becomes evaluation weights; the analyst adds a bid/no-bid recommendation with a win-probability score.
- **What stays:** the same three agents, the same checker, the same two-room handoff, the same human sign-off.
- **Demo:** 20-second montage — matrix, a mandatory certification the team would have missed, and a "conditional go" memo.

Deal Desk — the agency remix

Persona: a small design/dev studio. **Input:** a messy client brief + rate card + past projects. **Output:** scope, quote, invoice, kickoff pack, review.

- **What changes:** the matrix becomes a deliverables/assumptions matrix; the drafter writes the scope; the pricing tool computes hours × rate, margin and deposit; the output includes a quote PDF and a deposit invoice.
- **What stays:** the same three agents, the same checker (every deliverable addressed, every price cited to the rate card), the same approval gate.
- **Demo:** 20-second montage — "client brief → approved scope + invoice," with the same agents.

> SCOPE RULE
>
> Grant must be flawless. Bid is thin. Deal is thinnest. If time slips, cut Deal first, then Bid. Never cut the checker, the live demo, or the README. A polished single vertical beats three half-built ones.

Discord — primary demo

Discord is the best visual proof of Switch: each agent posts under its own name and avatar through a per-channel webhook, without a public URL. The setup is more involved than Telegram, but the demo is materially better.

1. Create a Discord server (free) and a Discord application + bot in the Developer Portal.
2. Enable **Server Members Intent** and **Message Content Intent**; save changes.
3. Invite the bot with `bot` + `applications.commands` scopes and permissions: View Channels, Send Messages, Send Messages in Threads, Manage Webhooks, Manage Channels, Manage Roles, Read Message History, Attach Files, Add Reactions.
4. Copy the server ID (Developer Mode → right-click server → Copy Server ID) and the bot token.
5. In Switch Console → server Home → Messaging apps → Connect → Discord. Paste token + guild ID. Agent-name autocomplete is a Slack feature; on Discord a Switch mention renders as plain text — expected.
6. Link your Discord account to your Switch user. Do it before the run: an owner-only agent reads an unlinked account as a stranger. Invite the teammates to the server.

Slack — second connection

Slack is the enterprise-looking proof that the same room works in another app. The docs require approval to install a custom app; a free personal workspace gives you admin, so this is feasible without an IT ticket.

1. Create a free Slack workspace (or use one where you are admin).
2. Create the Switch Slack app from the docs' manifest; install it into the workspace.
3. In Switch Console → Messaging apps → Connect → Slack. Link your Slack account too; the Reviewer is owner-only in both apps.
4. Create a Slack room and invite the same three agents. Do not re-run the template; that would try to create duplicate agents.
5. Record a 10-second clip of the analyst responding in Slack. Cut it into the demo.

> BRIDGE BEHAVIOR
>
> The template's `bridge` param defaults to `[Discord, Slack, $first]`. The first run creates the Discord rooms. For Slack, create a room manually and invite the existing agents — this proves portability without duplicating agents or rooms.

Fallbacks

IF THIS FAILS	FALLBACK	IMPACT ON SUBMISSION
Discord bot setup	Use the bundled Mattermost for the smoke test, then Telegram for the live demo.	Lower visual polish; workflow intact.
Slack admin	Document the Slack setup in the README; keep Discord as the only live bridge.	Lose the 10-second Slack cut; no impact on Track 01.
Windows Switch Console	Run Switch core via Docker Compose and use the Gateway; demo in Mattermost.	More setup friction; still Switch-native.
OpenCode provider	OpenCode is installed and configured; test Claude Code as the fallback (it may need WSL on Windows).	None; the template already supports it.

Team roles

PERSON	ROLE IN THE DEMO	WHAT THEY DO	WHAT THEY NEED
You	Approver / owner	Run the template, make the fiscal-sponsor call, approve fixes, sign off.	Switch Console, Discord, Slack.
Teammate A	Client / program director	Post the guidelines, answer the analyst, react to the matrix.	Discord/Slack account; no Switch install.
Teammate B	Compliance observer	Challenge the scorecard, flag the missing attachment, add pressure.	Discord/Slack account; no Switch install.
Teammate C (optional)	Finance/ops	Appear in the Deal Desk montage; react to the quote/invoice.	Discord/Slack account.

Recording window: Nov 6, 2 hours, suggested 4–6 PM IST. The host records the Discord window with OBS or Windows Game Bar; teammates join from their own devices. Record the Slack clip separately. Keep a backup recording of the deterministic `signoff demo` run.

Recording run of show

TIME	WHAT HAPPENS	OWNER
Before Nov 6	Lock the 4–6 PM window, send the invite, test audio in Discord, seed the sample files and channel messages.	You
4:00–4:20	Rehearse intake only; check zoom, cursor, notifications and the client's audio. Do not record yet.	You + A
4:20–5:10	Record the Grant flow twice. Keep the take with the cleanest checker catch and human decision.	You + A + B
5:10–5:40	Record the 10-second Slack clip and capture Bid/Deal stills for the montage.	You
5:40–6:00	Watch back once, export 1080p, upload unlisted, and paste the link into the README draft.	You

> PRE-FLIGHT, 10 MINUTES BEFORE RECORDING
>
> Analyst online in intake; all three online in draft; Console stays open (local agents stop when it quits). Both accounts linked; `!agents-status` clean in both rooms. Notifications off. Browser at 110% with no personal DMs visible. The deterministic `signoff demo` already recorded as a backup. If any of these is false, fix it before you press record.

07 Execution plan

THREE DAYS, FOUR EXIT CRITERIA

DAY	FOCUS	TASKS	EXIT CRITERION
Nov 4 Today	Setup spike + schema	Start Docker; install Switch Console; local server; test OpenCode agent in Mattermost; connect Discord and link your account; register on WeMakeDevs; create the repo and clone it to the workspace; commit template skeleton, vertical schema and checker spec.	One agent answers in Discord; `signoff demo` produces a sample matrix and scorecard.
Nov 5 Build	Core engine + Grant	Toolbelt (`extract` , `matrix` , `check` , `score` , `export`); Grant sample pack; three agent briefs; two rooms + link; happy path; rough recording; create Slack workspace and connect Slack.	End-to-end Grant run: guidelines → matrix → draft → review → DOCX.
Nov 6 Polish	Bid + Deal + demo	Thin Bid and Deal packs; pricing tool; approval gate; room link; clean-clone test; README + screenshots; final recording with teammates; Slack clip.	Clean clone + template import + 3-minute video recorded.
Nov 7 Ship	QA + submit	Fresh-venv test; README proofread; video captions; submit by 6 PM IST; publish blog; post #HowISwitch; verify submission confirmation.	Submission confirmed with 5+ hours of buffer.

First 90 minutes in Build mode

1. **Start Docker Desktop** and wait for "Docker is ready." If it prompts for WSL updates, handle that first.
2. **Install Switch Console** from the latest Windows x64 release. Open it; opt out of telemetry if you prefer.
3. **Run a server on this computer.** Wait for the first-start image download. Sign in to the bundled Mattermost.
4. **Create a test agent** with OpenCode (or Claude Code if OpenCode's provider is not connected). Ask it one question in Mattermost.
5. **Connect Discord.** Create the bot, enable intents, invite it, paste token + guild ID, link your account. Ask the same agent one question in a Discord channel.
6. **Create the repo** `signoff-switch` under `varun2507027108-oss` (public). Commit the template skeleton, `verticals/grant/rubric.json` , and `src/signoff/cli.py` with the `demo` command stub.
7. **Prove the toolbelt:** install the CLI into the workspace clone (`.venv` plus a `signoff.cmd` shim), run `signoff demo --vertical grant` , and open the matrix and scorecard.

> STOP CONDITION
>
> Do not start writing Grant content until step 5 works. If the agent cannot answer in Discord, we fix the bridge first. A beautiful workflow that cannot be demonstrated live is not a submission.

What we freeze, and when

- **End of Nov 4:** the checker schema and the template shape. No redesign after this.
- **End of Nov 5:** the Grant happy path. No new Grant sections after this.
- **End of Nov 6:** the demo video and README. Only bug fixes and captions after this.
- **Nov 7, 6 PM IST:** submit. No "one more feature."

08 Risks and fallbacks

ASSUME THE DEMO WILL FIGHT BACK

RISK	LIKELIHOOD	IMPACT	MITIGATION / FALLBACK
Windows Switch Console is early access	Medium	High	Setup spike first. If blocked, run Switch core via Docker Compose and use the Gateway; demo in Mattermost.
Docker daemon won't start	Medium	High	Start Docker Desktop before anything else; handle WSL update prompts; reboot if needed. This is the first 90-minute blocker.
OpenCode provider not connected	Medium	Medium	OpenCode is installed and configured; test Claude Code as the fallback (it may need WSL on Windows).
Owner-only reviewer can't recognize the approver	Medium	High	Link Discord and Slack before the run; ask the reviewer one question in each room; `!agents-status`; Console warns when the account is unlinked.
Discord bot permissions/intents	Medium	Medium	Use the exact intent + permission checklist; smoke-test in Mattermost first; Telegram fallback.
Slack app approval	Low–Medium	Low	Free personal workspace; if blocked, document Slack and keep the Discord demo.
Agent invents evidence	High without guardrails	High	Citation checker + evidence index + `[GAP]` rule + reviewer cannot approve. Demo the checker catching one.
Live demo failure	Medium	High	Pre-record; seeded sample data; deterministic `signoff demo` backup; second recording if needed.
Template creation fails	Medium	Medium	Fallback to manual Console setup + a room template; README includes a `setup.md` with exact steps. Reproducibility survives.
Teammates unavailable	Low	Medium	You + one friend as client; the workflow still shows two humans. They confirmed Nov 6, so lock the time early.
Scope creep across three verticals	High	High	Grant flawless; Bid thin; Deal thinnest. Cut Deal first, then Bid. The checker and demo are non-negotiable.

If the setup spike fails

CONSOLE FAILS, DOCKER WORKS

Run the Switch core server with Docker Compose, open the Gateway, create the workspace and connect Discord there. Use the same template YAML via the Gateway's template upload. The Console is a convenience, not a dependency.

DOCKER FAILS

Do not burn the day on Windows virtualization. Try a reboot and Docker Desktop's troubleshooting once. If it still fails, pivot the demo to a hosted Switch server or the bundled Mattermost on a teammate's machine; keep the repo and video as the primary artifacts.

09 Submission checklist

WHAT MUST BE TRUE BEFORE 6 PM IST ON NOV 7

Registration and team

- [ ] WeMakeDevs account created and hackathon registration completed.
- [ ] Team members added (2–4); each has agreed to the demo and submission.
- [ ] Switch repo starred (keyboard draw) and Discord community joined (support).
- [ ] Discord and Slack accounts linked; the reviewer answers the approver.

Repository

- [ ] Public GitHub repo `signoff-switch` under `varun2507027108-oss`.
- [ ] MIT license; no secrets, tokens, personal data or proprietary sample content.
- [ ] README with: one-liner, demo video, screenshots, architecture, "Why Switch is central," quickstart, judge's 5-minute path, repo layout, limitations.
- [ ] `signoff.template.yaml` imports cleanly; `workspace` points at the repo clone and the CLI runs there.
- [ ] Three sample packs (Grant, Bid, Deal) with fictional data.
- [ ] `pytest` passes; GitHub Actions CI green.
- [ ] `signoff demo --vertical grant` produces artifacts from a fresh clone.

Demo video

- [ ] 3 minutes max, 1080p, captions on.
- [ ] Shows Discord live: client, analyst, drafter, reviewer, approver.
- [ ] Shows the human decision, the approver waking the reviewer, and the sign-off.
- [ ] Shows the checker catching a missing citation or attachment.
- [ ] Shows the final DOCX and the decision log.
- [ ] Includes a 10-second Slack cut and a 20-second Bid/Deal montage.
- [ ] Uploaded unlisted to YouTube or Loom; link in README and submission.

Submission

- [ ] Repo URL, demo video URL, README URL, team names, track selection.
- [ ] Submission completed by **6 PM IST, Nov 7**; confirmation screenshot saved.
- [ ] Blog published (Dev.to or personal blog) with repo + video links.
- [ ] #HowISwitch post published on X/LinkedIn with the clip and repo.

Final QA (30 minutes)

1. Clone the repo into a fresh folder. Create a fresh venv. `pip install -e .`.
2. Run `pytest -q` and `signoff demo --vertical grant`.
3. Import the template into a clean workspace; run Grant end to end.
4. Open the README on mobile: does the hero, video and quickstart read clearly?
5. Check every link in the README and the submission form.
6. Submit. Do not wait for 11:59 PM.

Bonus entries

Best Blog — AirPods 5

Title: "Building Signoff: a human-in-the-loop compliance room with Switch."

Length: 1,200–1,800 words. **Platform:** Dev.to (fast, public, easy to submit). **Time:** ~2 hours, reusing the README.

OUTLINE

1. The problem: a grant deadline, a 12-page funder call, and a missed attachment.
2. Why a chat bot is not enough; why the room is the workspace.
3. The three agents and the permission model.
4. The deterministic checker: citations, gaps, word limits, attachments.
5. The human moment: the fiscal sponsor audit decision.
6. What broke during the build and what we changed.
7. How to reproduce it in 10 minutes.
8. What we would build next (Bid/Deal templates, more verticals).

#HowISwitch — keyboard draw

Post copy:

> We built **Signoff** for the Flip the Switch hackathon: three agents turn a funder's guidelines into a compliance matrix, a cited draft and a review scorecard — and a human makes the call. The checker catches every uncited claim before it reaches the reviewer. #HowISwitch @SwitchAI @WeMakeDevs [repo] [video]

Attach a 20-second clip: the client drops the guidelines, the analyst posts the matrix, the reviewer flags the missing attachment, the approver signs off. Post on X and LinkedIn; the hackathon counts the hashtag post.

Code with Wispr — $500

Only enter if you already use Wispr Flow. The side track requires a real Insights scorecard screenshot; do not fake it. If you have it, dictate the agent instructions and the README while building, then attach the scorecard to the submission. If not, skip it and put the time into the blog.

11 Appendix

LINKS, COMMANDS, DECISIONS, OPEN ITEMS

Source links

RESOURCE	URL
Hackathon page	wemakedevs.org/hackathons/switch
Switch docs	docs.switchagents.ai
Switch quickstart	docs.switchagents.ai/switch-rooms/getting-started
Template language	github.com/sandbox-quantum/switch/blob/main/docs/TEMPLATES.md
Discord setup	docs.switchagents.ai/switch-rooms/deploy/messaging-apps/discord
Slack setup	docs.switchagents.ai/switch-rooms/deploy/messaging-apps/slack
Switch repo	github.com/sandbox-quantum/switch
Console download (Windows)	github.com/sandbox-quantum/switch/releases/latest/download/switch-console-x64.exe

Commands

```
# repo
git clone https://github.com/varun2507027108-oss/signoff-switch.git
cd signoff-switch
python -m venv .venv
.venv\Scripts\activate
pip install -e .

# tests
pytest -q

# deterministic sample (no LLM)
signoff demo --vertical grant

# live verticals
signoff matrix --vertical grant
signoff check  --vertical grant
signoff score  --vertical grant
signoff export --vertical grant
```

Repo review: what changed

- The two-room shape matches Switch's own `support-desk` template — a greeter in both rooms, a specialist in one. Architecture validated.
- The Reviewer is `owner`-only: the approver must wake it. The drafter's "hand to the reviewer" line was wrong and is fixed. Agent briefs are vertical-agnostic, so reuse works.
- Identity claims are a hard gate: an unlinked account reads as a stranger to an owner-only agent.

Decisions log

DECISION	CHOICE	WHY
Track	Track 01 — Best Business Workflow	Highest ceiling; real artifacts; clear ROI story.
Concept	Signoff: one engine, three verticals	All three directions without three separate builds.
Hero vertical	Grant Room	Fresh, emotional, document-heavy, easy to verify.
Platforms	Discord primary; Slack second	Discord shows agents' names/avatars; Slack proves portability.
Team	2–4 humans; 3 agents	Real separation of duties; client vs approver.
Time	Full-time, 3 days	Enough for a polished Grant flow and thin Bid/Deal remixes.
Track 03	Skip as primary	61 open hackathon issues, all connector or provider integrations; maintainers assign every interested contributor and pick one PR per issue. $100 vs $2,000.

Open items

- Confirm the exact 2-hour recording window on Nov 6 (suggest 4–6 PM IST).
- Create the Discord server + bot, invite the teammates, and link your Discord and Slack accounts.
- Create the free Slack workspace; install the Switch Slack app.

Immediate next actions

ACTION	WHEN	TIME
Register on WeMakeDevs and create the team.	Today	15 min
Start Docker, install Switch Console, get one agent answering in Mattermost and Discord.	Today	90 min
Create the Discord server + bot, invite the teammates, test the client account.	Today	30 min
Create the public repo and commit the template skeleton, schema and demo stub.	Today	30 min
Confirm the Nov 6 recording window; create the Slack workspace and install the app.	Nov 5	20 min

> THE ONE THING TO PROTECT
>
> The demo must show a human making a decision the agents cannot make. If we lose that moment, Signoff becomes "three bots write a grant." The fiscal-sponsor audit decision is the heart of the story: the analyst surfaces the disqualifier, the human chooses the path, the reviewer verifies the result.